Your Data, Your Rules: Extending the Decentralized Web

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

Email decentralized where your data *goes*: a global identifier, sovereign provider stores, and standardized transport—no central authority, no shared mailbox. It never decentralized who may *access* it: the payload crossed the boundary; the permissions did not. Every social platform inherits that split at larger scale, building its feed, follow, and like as bespoke APIs behind a platform-owned backend. **web10** closes the split with a deployed, user-addressed data platform. Its node is completely generic — read and write on data, and configuration for identity and access, executed every time the data is used — and the access rules are the user's own data, in the very store they protect. Application-written queries pass a boundary the node compiles against the reader's grants: the snake game reads the social application's `profile` through a per-service grant, far more fine-grained than OAuth 2.0, with no node change and no agreement between the applications. Across nodes, identity is *verified, not registered*—a cached handshake like email confirming a sender—and an address is a domain the user owns, which the node verifies but cannot control. At the bottom is ClickHouse, doing real-time analytics at scale. The first measurement found the cost in the access layer, not the database (7.64 s to 363 ms at 1,000 followed creators), and the node is readable by design: data policy, not privacy. When the data leaves a node, the rules leave with it.

1 INTRODUCTION

Email is the decentralized web's proof of concept. A global identifier (`you@provider`), a sovereign store per provider, and standardized server-to-server transport let a message cross providers with no central authority and no shared mailbox database. Fifty years of practice show the model works at global scale.

What email never decentralized is *access*. You can email a Yahoo user from Gmail, but you cannot grant that Yahoo user *editor* on your Google Calendar: the payload crossed the boundary, the permissions did not. Every social platform inherits the same split at larger scale, and every one of them builds its features as bespoke APIs—a feed endpoint, a follow endpoint, a like endpoint—behind a platform-owned backend, so the content is portable in principle while who follows whom, who belongs to which group, and who may read or contribute stay locked inside the platform. The social graph—the follows, the groups, the audience—is what makes a person's data worth owning, and it is the part no platform lets the user keep.

web10 closes the split. It is a deployed, user-addressed data platform, and this paper is about the system that was built and is running: across the project's text files, all commits since November 2021, merge commits excluded, **958,824 lines were added and 568,594 deleted**. The node that hosts it is completely generic—read and write on data, and configuration for identity and access, executed every time the data is used—and the access rules are the user's own data, in the very store they protect. A social application, a snake game, and a marketplace run on the node and interoperate on shared data through the same boundary; the game and the marketplace read the social application's `profile` documents with no node change and no API agreement between the applications. Email let the data leave the building; web10 lets the *permissions* leave it too.

The contribution is a combination, and each part has a home in prior work; no prior system we compare carries the conjunction. Five properties, and this paper closes on each:

(1) **User-owned, portable data, all formats.** A post, a profile, a game score, and a marketplace listing are documents in the same store, differing only in the service string and the JSON shape the application agreed to use (§3).
(2) **Fine-grained, user-managed access.** A grant is a row: principal × group × service × operation; roles are per-service permission maps; publicness is a grant, not a flag. The end user runs the IAM operations a cloud operator would express as policies, over their own data (§2).
(3) **Cross-node-addressable access.** A principal is `provider/username`, globally unique across the federation; the policy reaches across the network, and a foreign token is verified, not registered—one cached `/certify` answer, the same access check on every read (§4).
(4) **Composable, app-defined queries.** The node has never learned a social concept—no posts table, no feed endpoint, no followers construct—and the query boundary keeps the shared store sound: the reader's readable set is compiled into the query before any data moves (§5).
(5) **A user-addressed network without federation.** Sovereign nodes, client-mediated reads, no central authority, no shared database; an address is a domain the user owns, and the audience travels with it when the account moves (§4).

The evidence is connected, bounded, and named (§8): the architectural claim (the boundary holds—the exploit cases replayed and denied), the business claim (a game and a marketplace reuse the social application's profiles through real browser consent and cross-author controls), and the scale claim (where the cost lives—the access layer, not the database: 7.64 s to 363 ms at 1,000 followed creators, with the one-group counterweight in the same trial).

This paper makes three contributions:

(1) **A user-addressed network model** (§4): a federation of sovereign nodes in which the user's client mediates cross-node reads, each node re-authorizes every read against a verified token, and the access model is globally addressable by construction— generalizing email from one data format to all formats and from content routing to access control.

(2) **A reader-specific query boundary** (§5): application-written queries over a shared columnar store, in which service references resolve to reader-filtered relations *before* the caller's joins and aggregation—the access decision is made on the input, not patched onto the output.
(3) **A deployed platform with a measured cost structure** (§6–§8): a social application, a game, and a marketplace over the same node, with connected browser consent and revocation; an isolated two-node authorization experiment; a domain-subscription prototype that outlives its provider; and the metadata fan-out measurement that drove the access layer's batching decision.

Figure 1 draws the system the paper walks through: the application's data layer in the browser, the thin generic node, and its two stores—with the messages that move between them. §2 gives the access model, §3 the data plane it protects, §4 the network model that makes it a federation, and §5 the boundary that enforces it. §6 shows what runs on the node—the social application, the two interop applications, and the ad rail the store inherits; §7 records what running it taught us, and §8 reports the evaluation—cross-application and cross-node correctness, domain-subscription continuity, the metadata fan-out cost, and a deployed operating point. §9 positions the design against the systems it combines, and §10 concludes.

2 THE ACCESS MODEL

The second property of §1 is the one this section must earn: the access rules are the user's own data, in the very store they protect. The permission model belongs to the user, not the platform. The model is small enough to state in one page, and it is the whole argument's forcing move: because a grant is a row, the reader's readable set is computable *before* any query runs—which is what makes the boundary of §5 a wall instead of a filter.

2.1 The vocabulary

For a group-scoped content operation, the model names an *app* (the calling application), a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two questions, at two different walls:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Does membership or a class/role grant allow O on S in G?

The app contract expresses trust in a calling application. In the current document and query paths, an authenticated browser-style token is checked against an origin-keyed contract when an HTTP `Origin` header is present; without that header the app-contract check is skipped, as it is for anonymous queries. OAuth-delegated tokens instead select a `client_id`-keyed contract, including when no Origin is present. The node enables wildcard, non-credentialed CORS; endpoint code enforces application grants separately. Origin identifies the browser's calling context, the delegated token carries the OAuth client identity, and non-browser clients control their own Origin headers. Identity verification and document/-group authorization are separate checks.

The group layer expresses trust in people. The split is deliberate: a user can revoke an app grant without leaving a group, or leave a group without touching app grants. Two walls, two kinds of trust, two independent revocations.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map has *the same shape* as the app-contract permission object: there is one permission language across both trust walls. Second, a person holds *one explicit member role* per group. The obvious alternative is multi-role stacking, where a principal accumulates roles and their permissions union. The per-service map makes that unnecessary: compound permissions are placed in the single role rather than stacked through multiple assignments, so there is exactly one role to inspect, explain, and revoke. Policies use positive grants; explicit deny and context-dependent conditions are outside this model. A group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`—or `dj`, `listener`); applications supply their role definitions, and some lifecycle operations use conventional names such as `member`.

Extending the platform costs a role and a policy—the seventh sentence of §1, stated as the model: a new application gets an app contract, and a user grants it per-service reads over their groups. It is the same job OAuth 2.0 does for cross-platform data sharing, but the grant names the service, the group, and the operation (§6.4).

2.3 Principal classes and the effective-permission expression

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	a verified non-anonymous user, member or not
`member`	holds a member row in this group

The broad classes are represented by *reserved keys in the member table*; individual members use their own principal keys. There is no separate visibility table, because visibility *is* membership-shaped data: a public group simply has an `anyone` row.

```
group_members:
  (G, 'anyone',        'reader')    -- public
  (G, 'authenticated', 'reader')    -- signed-in
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

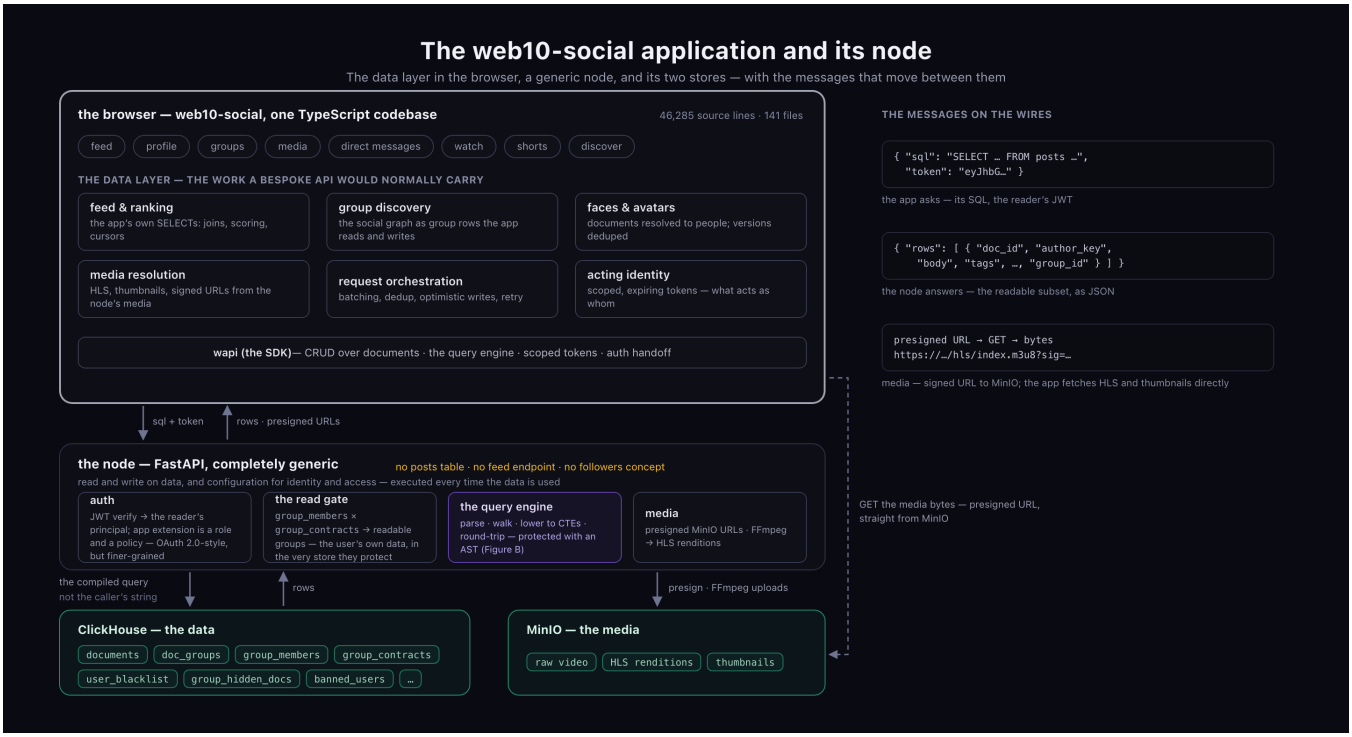


Figure 1: The web10-social application and its node—the data layer in the browser, a generic node, and its two stores, with the messages that move between them. The browser frame holds the application's surfaces and the six data-layer modules a bespoke API would normally carry: feed and ranking (the app's own queries: joins, scoring, cursors), group discovery (the social graph as group rows the app reads and writes), faces and avatars, media resolution, request orchestration (batching, dedup, optimistic writes, retry), and acting identity (scoped, expiring tokens— what acts as whom)—plus the SDK. The node frame is thin and generic: authentication, the read gate, the query engine, and media (signed URLs to object storage; the render pipeline to HLS renditions). The stores are drawn with their real names: ClickHouse's `documents` and group tables, and the object store's media keys. The messages panel shows the payloads that move: `sql + token` down; the compiled query—not the caller's string—and rows with signed URLs back; then the signed URL, the GET, and the bytes. Pinned: 46,285 lines of application-local source across 141 files (§6.2).

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;&=\; \text{grant}(G,\ \texttt{anyone}) \\ \cup\ &\big(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset\big) \\ \cup\ &\big(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset\big)\end{aligned}$$

This is the paper's central expression, and the place where the theorem starts: the readable set of a query is a set expression over rows that exist in the store, so it is computed from data the user owns, and it can be computed *before* the query touches any document (§5).

The content gate combines this grant union with membership admission. In the `can_read_group` and `can_write_group` helpers, an active member is admitted for content reads and creates without requiring service-specific `readAll` or `create` in that member's role; for non-members, the class grants supply those permissions. Other role-controlled operations use their respective permission checks. The union is monotone across principal classes under otherwise equal conditions, and blocks, sharing state, hidden documents, and node moderation apply additional visibility restrictions to the resulting content.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" withholds those non-member grants. A visibility flag would be a second access mechanism beside the grant table— two sources of truth that would have to agree. Making publicness a grant leaves one mechanism, and it composes: a group can be invite-only to *join* and still publicly *readable*—the discovery board is exactly that, because join policy (open, request, invite-only) controls how a human becomes a member and says nothing about what a non-member bystander can see.

This is the sense in which web10 brings IAM to the front. The operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user*

expresses about their own data, over the same vocabulary. A grant is represented by metadata rows and revoked through tombstones; subsequent checks use latest-row resolution. End-to-end revocation latency therefore comprises database visibility, concurrent checks, in-flight requests, and the remaining lifetime of issued capabilities—each a separate term, each measured or bounded where this paper measures it.

3 THE DATA PLANE: ONE TABLE PER NODE

The access model of §2 is only as good as the store it guards. At the bottom of the system is ClickHouse [17]— an open-source columnar engine built for real-time analytics over billions of rows, in a way that scales—and the platform's answer is deliberately unglamorous: on each node, a user's content and the platform's access metadata live in *one* columnar database. ClickHouse serves both structured content and operational metadata, rather than a Postgres-to-ClickHouse replication pipeline feeding a separate operational store. The user namespace is *logical* isolation within that table—not a per-user database, not a physical collection. Large media objects reside in object storage; documents reference them. The simplified schema, omitting reference and advertising fields:

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
        comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

The `collection_name` column *is* the service: a post, a profile, a game score, and a marketplace listing differ only in that string and in the JSON shape the application agreed to use. Applications are free to define new services without a schema change to the platform—the first property of §1, at the level of the store.

3.1 Updates are inserts; deletes are tombstones

Updates insert a newer version; deletes insert tombstones. Background merges alone do not establish read correctness: reads must select the latest version *including tombstones*, and only then filter deleted rows. Filtering tombstones *before* deduplication can resurrect the previous version— the classic ReplacingMergeTree trap, and one the boundary compiler of §5 is built around. Documents attach to groups through `doc_groups`; `group_contracts` and `group_members` hold the contracts and assignments of §2. These are narrower metadata tables whose size grows with groups and memberships, not with document volume.

3.2 Why not a database per user—the composition fork

The database literature on multi-tenant isolation—shared tables carrying an organization identifier [18], schema-mapping techniques [2], the shared-versus-separate spectrum [9]—places its tenants at the *organization*. web10 takes the spectrum to its limit: one logical tenant per *end user*, in a shared columnar store. Physical isolation was never what the access model needed. The isolation a reader requires—"show me only what I may see"—is a property of the *query*, not of the container, and §5 supplies it at the query boundary.

The nearest prior work takes the opposite limit: one store per user, and the data optionally encrypted and pod-bound. Solid [15] is data-sovereignty first—your data in your own pod, and you control who can read it. The fork between the two designs is compositional, and it is the difference the scale of the store makes. In Solid, each user's data lives in that user's own pod, so a cross-user query must pull the whole network, and it does not terminate at scale: there is no shared index to run "who is trending?" against. A web10 node pools millions of users in one shared store, so discover, trending, and ranking are just queries the application writes against it—cross-author relational composition inside a single analytical engine, at application scale, and with no cross-database synchronization dependency that every replicated two-store design must pay for. Figure 2 draws the two worlds.

The costs of the shared table are real and are itemized where the running platform meets them (§7): version accumulation and merge pressure in the columnar store; read-time deduplication on every read; point-query latency paid to an engine built for scans; shared-resource contention between tenants in one store; multi-row content/membership changes and concurrent revocation requiring explicit consistency handling; and backups that span database metadata and object storage.

3.3 Data policy, not privacy

web10 deliberately trusts the node operator with readable content: search, ranking, moderation, and audit operate on plaintext. The node is readable by design; operator-blindness is not a goal. Tokens and signed capabilities protect the application and user boundaries, and terms govern operator conduct. This trust model is what allows node-side querying at all, without requiring default end-to-end encryption.

The stance is worth stating plainly, because it is the opposite of the familiar decentralization bet. web10 is not a privacy platform: end-to-end private messaging already exists (Signal, WhatsApp), and web10 does not try to replace it. It optimizes for reach and two-sided ownership, so its data sits unencrypted and readable on the node—that is what makes the high-performance ClickHouse queries and the data-lakehouse composition of this section possible, and what keeps a platform that must face abuse auditable rather than operator-blind. The bet is that users do not care that the node operator can read their data: they own the control over who *else* can reach it—a grant the user gives is a row the user deletes—and the operator is bound by law, not encryption, not to tamper with it. What the user owns is the policy, not the secrecy: a creator reaches the audience at scale, and the audience owns its follows (§4).

Here *user-owned* means logical authorship namespaces and user-controlled grants. The operator retains physical access to hosted data; legal remedies depend on terms, jurisdiction, and enforceability; and usable export and exit are broader product goals. The isolated cross-node read and public domain-subscription slices demonstrated in §8 are evidence about specific paths; complete

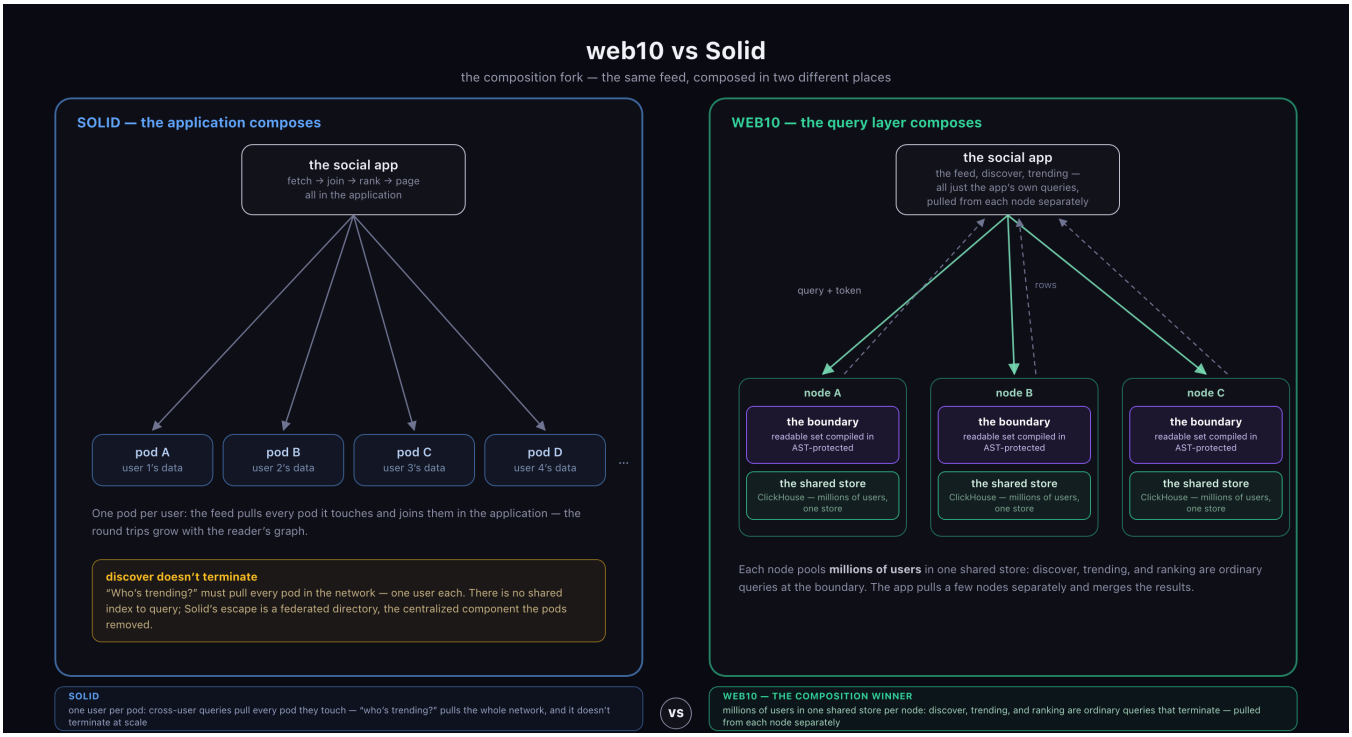


Figure 2: web10 vs Solid: where the composition happens—the same feed, composed in two different places. Left: Solid's one-user-per-pod store: a cross-user read pulls every pod the query touches, and a discover-type query must pull the whole network, so it does not terminate at scale. Right: three web10 nodes, each pooling millions of users in one shared ClickHouse store; discover, trending, and ranking are ordinary queries the application writes against it, each node re-checking the reader's rules on every read. The strip names the win: composition—a cross-user query that terminates (§5, §9).

account/data migration and production federation remain forward scope (§4).

4 A USER-ADDRESSED NETWORK

The access model of §2 is defined for one node. This section states the network model that makes it a federation rather than a single deployment—and the property that distinguishes web10 from both email and from server-federated systems like ActivityPub.

4.1 Email's architecture, generalized

Email combines globally addressed accounts (`you@provider`), sovereign provider stores, standardized inter-provider mail transport, and no central shared mailbox database. Its servers do exchange mail with other servers; client-mediated cross-node reads are web10's transport choice, not a property inherited from email.

web10 retains independent providers and global addressing. The identifier is `provider/username`—the provider is the node, the username is the user, and the pair is globally unique across the federation. The store is a shared columnar data plane (one `documents` table per node, keyed by `(author_key, doc_id)`; §3), not a per-user mailbox, but it is still sovereign: each node's store belongs to that node, and a user's home node is where their account and their grants live.

The two extensions of §1 are now precise. (1) The data is no longer locked to one format: any application defines its own services over the same store, so the network carries posts, profiles, scores, listings, and whatever an app names. (2) The *access layer* is decentralized, not just the data layer: a grant is a row `(group, principal, role)` in which the principal is the global `provider/username`. A user on node A can be granted `editor` on a group that lives on node B. The permission vocabulary of §2—principal class, service, operation, grant—is the same whether the principal is local or three nodes away.

4.2 The linchpin: verified, not registered

A user's home node is where their account lives. When that user reads content on another node, they do not sign up there. Their client presents their home-node token; the foreign node obtains home-issuer certification, resolves the principal to `provider/username`, and applies its own group-membership and grant checks. The user's identity is *verified, not registered*, on the foreign node. Email has already solved the analogous problem for messages: an arriving message is asked *is this sender who they claim to be?* (SPF, DKIM), and that check grants the message no access—the recipient's own filters still apply. `/certify` asks the same of an arriving token: one cached answer per token, and it verifies and grants nothing—a handshake, not a registry, because

every node still re-checks its own rules on every read. Same shape, different mechanism. The consequences are the model:

- A user's following set can span N nodes; they hold exactly one account, on their home node.
- A node's data is read only by holders of a verified token that the node's own membership and grant checks authorize—whether the token was minted by this node or a foreign one.
- "Following someone on another node" is a *cross-node membership*: a member row on the foreign node referencing a foreign principal. Content is fetched by the reader's client; server-to-server content replication is not required.

4.3 The boundary: the HTTP API and a verified token, never the database

A node never queries another node's store. The cross-node boundary is the *HTTP API* with a *verified token*; a foreign node's API call passes through that node's normal authentication and access gate, exactly like a local call. We reject database-level cross-server mechanisms (ClickHouse's `remote()`/`cluster()`) for federation: they are for a trusted cluster under one operator, they couple schemas across independent nodes, and they create a DB-level trust path that bypasses the API's authorization entirely—a node operator's database credentials would become a federation credential.

This yields the invariant the whole model is built to hold:

Cross-node boundary rule. The cross-node boundary is the HTTP API and a verified token; remote data is content, never control. A foreign node's response is untrusted content—the same category as a post body. It is never query input, never a key into the local store, and never a grant.

The rule prevents a remote response from serving as local access authority. The client merges independently authorized results as data, not as SQL to execute against another node. A data reference is not a grant: if B's content points the client at an A-node document, A checks that fetch against the reader's token and its own access policy. Foreign identity verification precedes, rather than replacing, the serving node's document-access checks.

4.4 The read model: client-side fan-out

When a user's feed spans nodes, the *client* makes the cross-node calls. It holds the user's home-node token, groups the user's following set by provider, and queries only the distinct nodes the user follows people on—targeted, not broadcast. Each node returns its own access-gated result set; the client performs a ranked merge. Figure 3 draws this path for one read: node B authenticates the home-node token over the issuer's `/certify` (one cached answer per token, never a grant) and then runs the same access check node A runs on local reads—contracts and membership, against B's own store, on every read. This model avoids a server-side content proxy and the additional credentials and content-cache policies it would require. Feed composition remains an application concern rather than a node-specific endpoint—a deliberate cost of keeping the node generic, and the reason the client does more work than it would against a conventional backend.

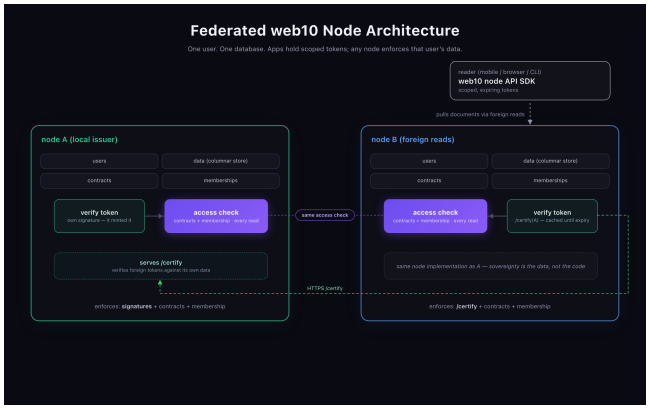


Figure 3: Pulling the web10 feed across nodes. The reader issues reads with its home-node token; when a read lands on foreign node B, B authenticates the token over the issuer's HTTPS `/certify`—one cached answer per token, never a grant—and then runs the same access check node A runs on local reads: contracts and membership against its own store, on every read. One node implementation, two sovereign stores; remote data is content, never control (§4).

4.5 Domain aliases and public subscription continuity

The `provider/username` principal is the address, but the username need not be an opaque string. Through the authenticator's claim flow, a user can bind a DNS domain to their account: a TXT record at `web10.domain` carries `web10.v=1; provider=node;user=username`, and an authenticated `POST /v3/domains/verify` request makes the node perform the DNS lookup itself, checking that the record's provider matches the node and its username matches the authenticated caller before storing the verified alias. The record is a pointer, not a credential: DNS lookup is discovery, not authentication, and the username remains the principal—groups, tokens, and the read gate never key on the domain. It is the same pattern the web already uses for identity claims (DKIM, SPF, `OpenPGPkey`), applied to a social identity.

This indirection preserves a *public* subscription when a creator changes accounts or providers. The concrete case: a creator's audience follows the domain `jacobhoffman.xyz`, not the `provider/username` the domain currently points to—which may be one account today and another tomorrow. The follow is data the *follower* owns: a private document in the follower's own store whose body is exactly the domain, written once. When the creator moves—new account, new provider—the creator re-points the TXT record, and the follows travel with the domain: the follower's client re-resolves the address on each read, and no platform holds a ban switch over an address the user keeps. The node verifies the record by performing the lookup itself, but the domain is untouchable to the node owner, who controls no part of it; Figure 4 draws the move. A compatible client therefore stores the domain—rather than a fixed account address—in the reader's own data. If the creator retains DNS control, the client can discover a newly

verified account without asking the former provider to approve the move. The subscription remains at the reader's home; neither the creator's content nor private access grants move through DNS. §8.3 demonstrates this path after the old creator API becomes unavailable, and says exactly how far the demonstration reaches: production web10-social follows remain group-based, domain control, resolver availability, and availability of the reader's home and new provider are assumptions rather than guarantees, and legal ownership and remedies depend on terms and jurisdiction, not on DNS resolution.

4.6 Implementation status and deployment scope

The implementation supports a bounded cross-node read path. Foreign readers use `{provider}/users/{username}`; local writers and readers retain legacy bare keys until a coordinated migration, and canonicalization is not complete across the identity layer. For a foreign token, the serving node asks its home issuer's HTTPS `/certify` endpoint and caches a successful identity answer for that exact JWT until expiry. The issuer verifies with its own signing secret; the serving node still checks its own access grants on every read. This live-certification design does not require JWKS or shared signing secrets. The isolated experiment in §8.2 uses independently generated node keys and owner-managed foreign grants. Production key provisioning, foreign self-join, complete principal migration, and client-side feed merging remain open. The current shared signing-key default must not be mistaken for independently provisioned production nodes.

5 ENFORCEMENT AT THE QUERY LAYER

Application-written SQL over a multi-tenant store requires a boundary *before* the caller's relational operations. The application's developer writes the query and tunes it; the platform guarantees the boundary, not the performance—the fourth property of §1, stated as the division of labor. The query API accepts a constrained read-only SQL surface, not arbitrary database access, and its intended boundary is a *wall, not a membrane*: validated service references read reader-filtered CTEs rather than raw tables whose output is filtered afterward. Correctness depends on validation coverage and SQL dialect interpretation, as discussed below.

Why output-side filtering is unsound. The natural design wraps the caller's query and filters the *result*— `SELECT * FROM (q) WHERE <filter>`. It fails because the caller's query has *already read the raw table* inside the parentheses, and an outer filter cannot undo an aggregation that baked everyone's data in:

```
-- counts every user's comments; the outer group filter
     is too late
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

The aggregate has already collapsed all authors into counts before any row-level filter could run. Filtering the output of an aggregation is not access control.

Redirecting the input instead. web10 parses submitted SQL to an AST and does not send that original string to ClickHouse. It validates table references and injects a *boundary CTE* under each referenced service name. The CTE deduplicates the service's documents, joins current `doc_groups` mappings, filters to readable groups, and applies block, sharing, hidden-document, and node-ban exclusions. This is the materialization of §2's central expression: the service name resolves to the reader-filtered relation of effective(P, G) *before* aggregation or joins run; the readable set can include public or authenticated-class grants, not just the reader's memberships, and an empty set produces an empty service CTE. Before compilation, an authenticated request with explicit groups is rejected with 403 if none of its referenced services has a readable group. The query boundary does not automatically union in all reader-owned documents. Figure 5 follows one feed query through all of it.

The walk and the backstop. Compilation is five steps: (1) parse to an AST (unparseable or multi-statement input is rejected); (2) check the statement kind (`SELECT` and supported set operations, not `INSERT` or `DROP`; this is not full SQL type checking); (3) walk every `Table` node, including inside caller CTEs and subqueries, rejecting raw node tables, ungranted services, and table functions (`file()`, `s3()`, `numbers()`, represented in the validated parser shapes as empty-named `Table` nodes); (4) inject the needed boundary CTEs ahead of the caller's CTEs and re-emit from the validated AST; and (5) *round-trip*—re-parse the compiled SQL and require it to come back as exactly one statement. The round-trip checks composition syntax, and targeted parser/engine tests exercise selected dialect and name-resolution cases. The executed query is emitted from the validated AST.

The OLTP-era answer to many users on one database was row-level security inside the engine: PostgreSQL's row-security policies [13] evaluate per-row policy expressions, functions, and subqueries inside the engine to consult dynamic authorization state. On a columnar store the boundary moves to the query, where the readable set is compiled in before any data moves—the sixth property of §1, stated as placement. web10 uses a shared database service account and constructs a reader-specific boundary combining membership, class grants, and visibility exclusions. This puts policy orchestration in the application rather than provisioning a database account and policy for each end user; an equivalent membership policy provides a comparison baseline for the cost of these placements, which §8 leaves as a matched workload to run.

A trusted implementation. The parser, validator, generated CTEs, identity resolution, metadata checks, and ClickHouse name-resolution semantics are in the trusted computing base. The service account must read raw tables to execute the generated CTEs; its privileges do not independently enforce each reader's boundary. Repository tests cover selected top-level, caller-CTE, subquery, table-function, and aggregation cases, with a live-node end-to-end gauntlet for supported shapes. Parser and engine upgrades require renewed compatibility and adversarial testing.

5.1 Metadata resolution and the relational cost of access

The read path is the heart of the design:

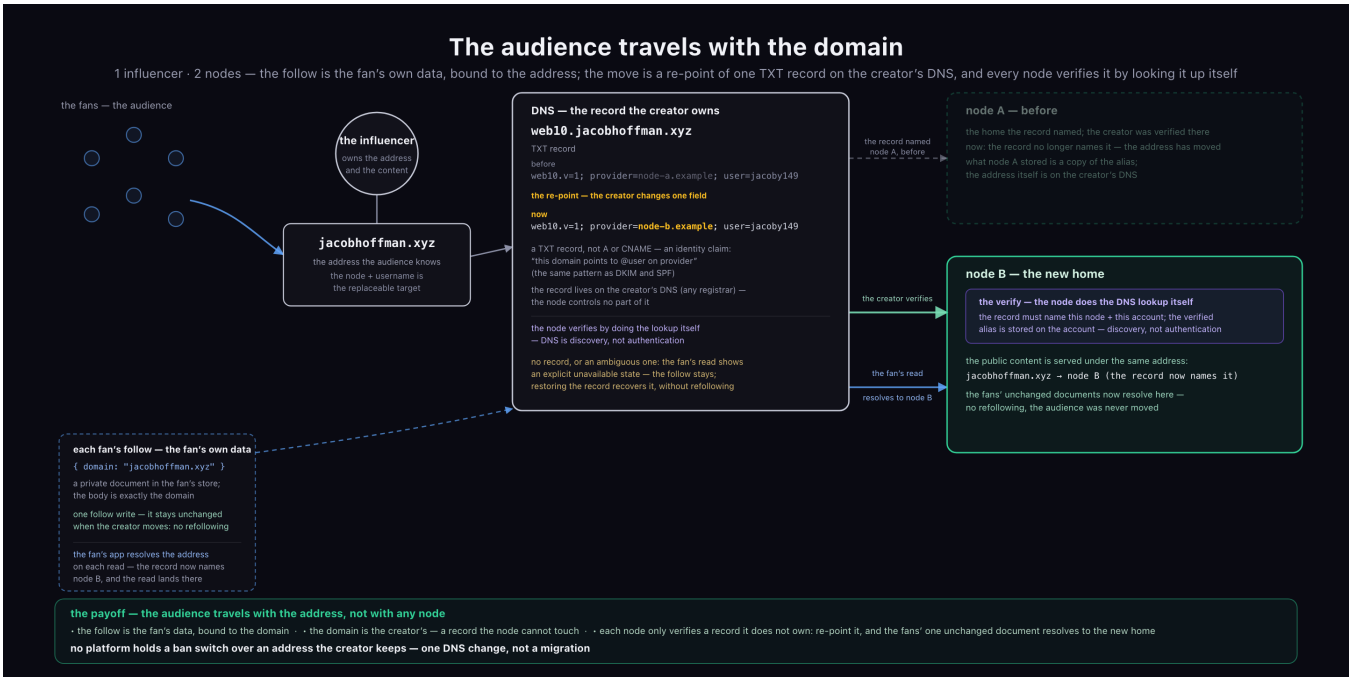


Figure 4: The audience travels with the domain: one influencer, two nodes, the DNS flip—the mechanism behind the demonstration of §8.3. The fans: each follow is the fan's own data—a private document in the fan's store whose body is exactly the domain; one follow write, unchanged when the creator moves; the fan's application re-resolves the address on each read. The domain chip: `jacobhoffman.xyz`—the address the audience knows; the node and username are the replaceable target. The DNS card, shown in both states: before `provider=node-a.example`, now `provider=node-b.example`—one field changed. A TXT record, not an A or CNAME record: an identity claim, the same pattern as DKIM and SPF; the record lives on the creator's DNS, and the node controls no part of it. A missing or ambiguous record gives an explicit unavailable state—the follow stays, and restoring the record recovers it without refollowing. Node A (faded): the home the record used to name; what it stored is a copy of the alias. Node B (active): the verify gate—the node does the DNS lookup itself, the record must name this node and this account, discovery not authentication; the content is served under the same address, and the fans' unchanged documents now resolve here. One DNS change, not a migration (§4).

(1) **Resolve.** Take explicitly requested candidate groups, or discover the reader's memberships. Parse the query to identify its services.
(2) **Compute.** For each referenced service and candidate group, check active membership or resolve class grants through member and contract metadata. The current query endpoint calls `readable_groups`, which loops over candidates; a check may issue several metadata queries.
(3) **Execute.** Inject each service's readable-group set into its boundary CTE. ClickHouse performs document/mapping deduplication, the mapping join, visibility exclusions, and the caller's joins, aggregates, and sorting.

For s referenced services and g candidate groups, this path performs up to sg group-gate evaluations, with early membership success reducing work within a check. The repository also has a batched helper used elsewhere; the query endpoint at the measured snapshot uses the per-group helper. Role resolution does not need content bodies, but its metadata I/O and the generated relational work both contribute to authorization cost—the question §8.4 measures.

The document-to-group join can yield multiple rows for one document attached to several readable groups. Applications must choose whether to retain those associations or deduplicate before counting and pagination. CTE expansion, join cardinalities, exclusions, and sorting can change database work and query plans; the evaluation accounts for both metadata resolution and this relational work.

5.2 Isolation rule and regression coverage

The intended isolation rule: another author's document requires an authorized group path, including public or authenticated-class grants. The query path implements this through the service CTEs. Permission-matrix and exploit-regression tests exercise the compiler and endpoint boundaries. CRUD, directory, group-attachment, preparation, and media paths have separate test scopes; §8.6 records the current coverage and open boundaries.

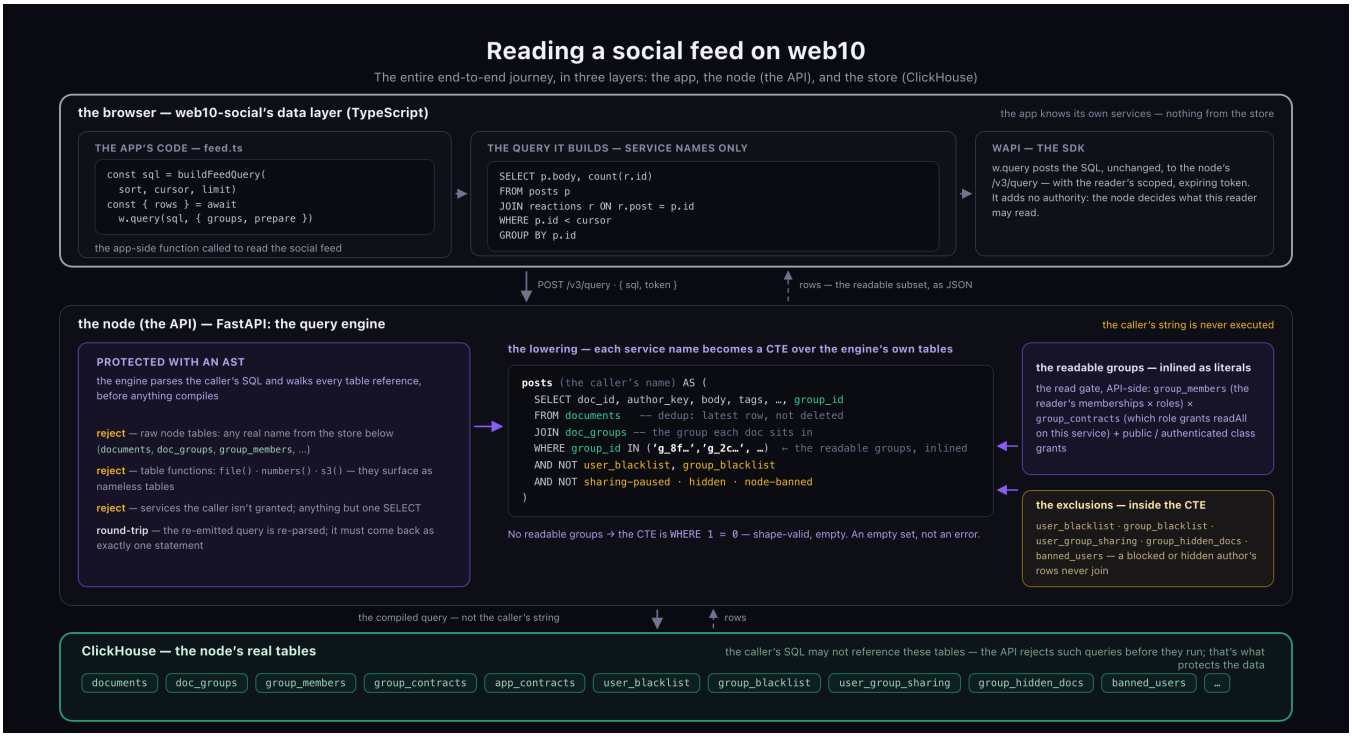


Figure 5: Reading a social feed on web10: the entire end-to-end journey, in three layers—the app, the node (the API), and the store (ClickHouse). Browser side: the app's feed code builds the query over service names (`posts`, `reactions`, `comments`, `profile`); the SDK posts `{sql, token}` to `POST /v3/query`—it adds no authority, the node decides what this reader may read. Node side: the API parses the SQL to an AST and walks every table reference—rejecting raw node tables, table functions, and ungranted services, and round-tripping the emitted SQL to exactly one statement—then lowers each service name to a boundary CTE over the engine's own tables: `documents` deduped to the latest not-deleted row, joined to the current `doc_groups` mappings, filtered to the reader's readable groups (inlined from the read gate), with the exclusion tables applied. The caller's string is never executed: the query that runs is emitted from the validated AST, and the ClickHouse strip names every real table the caller may not reach.

5.3 Document-linked media capabilities

Structured documents refer to binary objects in MinIO/S3-compatible storage. The intended linkage is document authorization followed by capability issuance: presigned URLs let browsers fetch individual objects directly, and HLS uses short-lived signed manifests and segment capabilities, with document checks on manifest access. Expiry limits capability lifetime but does not retract bytes already fetched or guarantee immediate revocation.

The query prepare pass combines media resolution, HLS signing, advertising, and avatar preparation, keeping capability issuance next to result preparation while object storage serves the media bytes. Its authorization boundary includes key ownership, authorized cross-user references, result projections, public thumbnails, and stale capabilities (§8.6).

5.4 Implementation boundary

Python/FastAPI handles request validation, identity and permission orchestration, SQLGlot parsing/compilation, serialization, and result preparation; ClickHouse's native engine performs scans, joins, aggregation, filtering, and sorting. The node API source and TypeScript SDK are available alongside the query endpoint and boundary compiler. The production API command configures four Gunicorn processes with Uvicorn workers. Profiling separates API CPU, database CPU, I/O waits, and queueing to locate the costs of parsing, metadata round trips, response serialization, and preparation.

6 RUNNING THE PLATFORM

web10-social is the demonstration application built on web10—and the platform's primary deployment. Its persistent social data and authentication use web10 APIs and the SDK rather than a separate social-data backend. Figure 1 is the division of labor this section runs: the data layer lives in the app, and the node stays thin.

6.1 The killer-app hypothesis

web10-social is also the project's *killer app*: an application compelling enough to attract users to a new platform. The longer-term

Table 1: Media and interaction patterns covered by the demonstration. Platform names identify familiar interaction patterns.

Demonstration surface	Familiar reference
Communities, text posts and discussions	Facebook, Threads
Image and mixed-media posts	Instagram
Vertical short-video browsing	TikTok
Long-form video watch pages	YouTube
Audio-track browsing and playback	YouTube Music

vision is an open, decentralized ecosystem in which many applications reuse a shared data and access substrate; social is the initial adoption vehicle and a demanding exercise of that substrate.

The application spans a media spectrum from text and social relationships to images, short-form video, long-form video, and audio. Facebook, Instagram, Threads, TikTok, YouTube, and YouTube Music provide familiar reference points for these interaction patterns (Table 1); the point is not parity with any of them. The point is that every one of these surfaces composes on the same document, group, and query substrate.

A successful adoption vehicle would bring those users and their relationships into the wider application ecosystem—which is exactly the condition in which the generic-node discipline of §5 starts paying, as §6.4 shows.

6.2 A frontend over the shared node

The application is a React/TypeScript SPA. Its data client connects to the node; social operations express posts, conversations, relationships, and feeds through web10's document, group, query, and media interfaces. It is backendless from the application developer's perspective: web10 supplies the backend, and the application supplies its data conventions, queries, and interface. The browser build is static assets and can be hosted independently, for example in S3 behind HTTPS/CDN with SPA route handling.

The current deployment also includes a small Node link-preview renderer for crawlers. That presentation component uses web10 APIs and holds no separate social-data store; a static-only deployment needs a separate arrangement for crawler cards. Media bytes transfer through API-issued object-storage URLs, and real-time peer traffic uses the SDK's RTC support; telemetry and external embeds are additional network paths.

In the measured source snapshot, the application-local frontend contains **46,285 physical source lines across 141 files**: 45,846 TypeScript/TSX lines and 439 CSS lines under `src/`, excluding tests. Its local tests contain another 31,060 lines in 105 files; the link-preview renderer is 431 lines in three JavaScript modules. Counts include comments, blank lines, and local type declarations; they exclude public/generated assets, dependencies, configuration, tooling, and sibling shared-library and SDK implementations. Implementation links use that pinned revision; the later experiment snapshots are identified separately in §8.1 and §8.4.

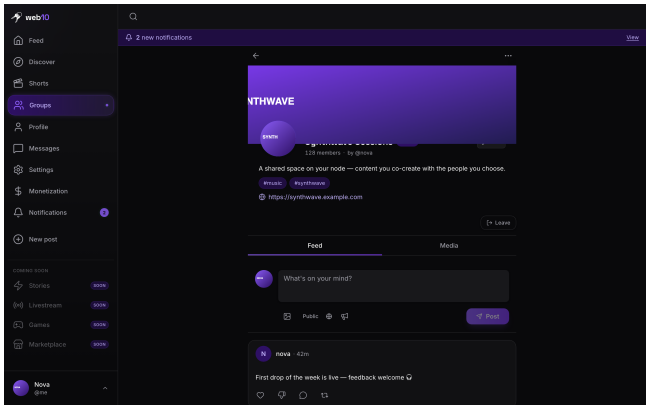


Figure 6: A group—"a shared space on your node." Membership, roles, and per-post visibility (Public / node / group) expose application choices over the generic groups and grants of §2.

6.3 The feed as a query

The application's feed implementation builds a query over `posts`, `reactions`, `comments`, and `profile`: it chooses chronological or weighted ranking, joins engagement aggregates and author profiles, and supplies cursor conditions. The node resolves those service names through the boundary CTEs of §5. Preparation can attach advertising and resolve media, HLS manifests, and author avatars on returned rows. This separates the application's feed definition from a dedicated feed endpoint, while retaining explicit server-side preparation machinery. Figure 6 shows the social surface the feed composes over: a group is a shared space on the user's node, and its membership, roles, and per-post visibility are the application's choices over the generic grants of §2.

Three implementation observations define what the feed path exercises:

(1) **Page assembly.** For a nonempty page with post IDs, `readFeedPage` issues the main query, then two engagement queries (reactions and comments) and a `contentViews` call. Group discovery precedes this work, and media bytes require separate fetches. Page assembly therefore combines one main composition query with scoped follow-up reads.

(2) **Authorization scope affects semantics.** The main query's engagement joins use the feed-group scope, while engagement documents may live in the discovery group. The follow-up queries use the union of feed groups and discovery to populate displayed counts, which are relative to readable relations, not necessarily global totals: the displayed counts can differ from the counts used by the main ranking query.

(3) **The query is application-defined.** Ranking and filtering can change without adding a dedicated node feed endpoint; applications select their ranking and projections while reusing the same boundary construction and preparation services.

The measurement in §8.4 isolates metadata resolution for a simplified query; it does not time this complete page-assembly path, and full-feed latency requires accounting for follow-up requests and media delivery as well.

Table 2: Data producers and consumers in the connected demonstration. Other social services are omitted.

Application	Writes	Reuses
web10-social	`profile`	—
Snake	`snake_scores`	`profile`
Marketplace	`marketplace_listings`	`profile`

6.4 Multiple applications, one data plane

The social application is a *producer* of reusable data, not the only client of the node. Two small static applications consume its `profile` documents: Snake associates completed runs with player identities; Marketplace associates listings with seller identities. Each writes its own service and reads the existing profile format (Table 2). Neither modifies web10-social, and neither adds a game, commerce, leaderboard, or seller-profile endpoint to the node. The common dependency is the document convention and the access primitives—not an application-specific API agreement.

Extending the platform is as simple as a role and a policy: the new application gets an app contract, and the user grants it the per-service reads it needs—the same mechanism OAuth 2.0 uses for cross-platform data sharing, but finer: the grant names the service, the group, and the operation. Figure 7 draws the fork this enables. On Mastodon and Bluesky, a second application reads through the platform's fixed endpoints or replicates the user's records into its own backend; a new read pattern is a new backend, and the object model has no record shape for a hi-score or a listing. Here, a second app gets a typed read on the first app's data and joins it in SQL on one shared store: a new read pattern is a new query.

The following abbreviated leaderboard query illustrates the seam. Both service names resolve to reader-filtered relations before the join; the full application also selects each player's best score and supplies profile-media preparation.

```
WITH latest_profiles AS (
  SELECT author_key, body FROM profile
  QUALIFY row_number() OVER (
    PARTITION BY author_key
    ORDER BY updated_at DESC, created_at DESC,
             doc_id DESC) = 1
)
SELECT s.author_key,
       JSONExtractInt(s.body, 'score') AS score,
       p.body AS profile_body
FROM snake_scores s LEFT JOIN latest_profiles p
  ON s.author_key = p.author_key
ORDER BY score DESC LIMIT 50
```

Permission does not leak through the join. An application's permission to read scores does not confer permission to read profiles, and an app-level profile grant does not itself grant access to a private profile group. The outer join can retain a score without a readable profile; the independent group control in §8.1 checks the corresponding inner-join exclusion while the score records remain readable. Applications therefore compose *authorized relations*, not unrestricted copies of another application's data.

Figure 8 shows the resulting player and seller identities. The synthetic name was edited through the social UI, then read by both applications; neither demo writes or recreates the profile. The connected checks in §8.1 verify the underlying records as well as their rendered names.

6.5 How money flows on web10: the ad rail

Solid, Mastodon, and Bluesky ship no ad mechanism—ad-free by conviction, a documented design property (§9)—so on those platforms the creator's only revenue path is a cut the platform takes. web10 takes the opposite position: the free web should have ads, and the ad architecture matters. The node does not take that position on payment: it processes no payment and takes no cut of the creator's revenue. It takes it on monetization, with the ad rail that Figure 9 draws.

Every application that writes into the store inherits the rail: a new ad surface is a new query, not a new backend. web10-social builds the ad catalog today; the snake game and the marketplace inherit it—any of their documents can be a pinned ad target or a node ad. The node attaches the ads at the boundary, checked the same way every other read is checked (§5), with the density split explicit. The creator's layer is dense: the influencer's own offers—an affiliate link, a sponsorship, the node's creator tier—land on most of their content, roughly 80–90% of the time at the competitive operating point. The node's layer is sparse: the operator's overlay lands on roughly 5–10% of posts, node-controlled, default 10, deterministic per reader, round-robin across the inventory. Both can be present on the same post, and the node's overlay never suppresses the creator's monetization. The inventory is tagged documents plus an offer object—kind, the link that pays, call to action, disclosure—that the platform never rewrites and never cloaks.

The money itself stays off-platform. The creator keeps 100% of affiliate and sponsor revenue: the link pays the creator, the platform takes no cut and processes no payment. The node operator is a media company: the overlay sells the node's own attention directly, and the operator keeps the bulk of it—roughly 85–90%. And because the overlay density is node-controlled, nodes compete for the influencers by lowering their rates; §9 draws that market. The horizon is separate from the claim: an ad exchange, a payments rail, and the creator's own layout sit ahead of this system, and the figure draws them as a distinct layer.

6.6 Durability and extensibility

Application-named services let different clients interpret documents without changing the shared document schema. A conversation can be a group with message documents; a music library can define track services and listener roles; a shop can define catalog services. Domain-specific transactions, payment workflows, search indexes, schema evolution, and external integrations can extend this common substrate.

The durable separation is between stored content/access relationships and a particular frontend. Replacing a frontend can reuse those primitives when it understands their semantics. The user-addressed network model of §4 is the architectural answer to provider migration: the user's principals and grants are globally addressable. Global addressing does not itself migrate an account, its content, or its grants. The public domain-subscription prototype exercises one continuity mechanism; complete account/data

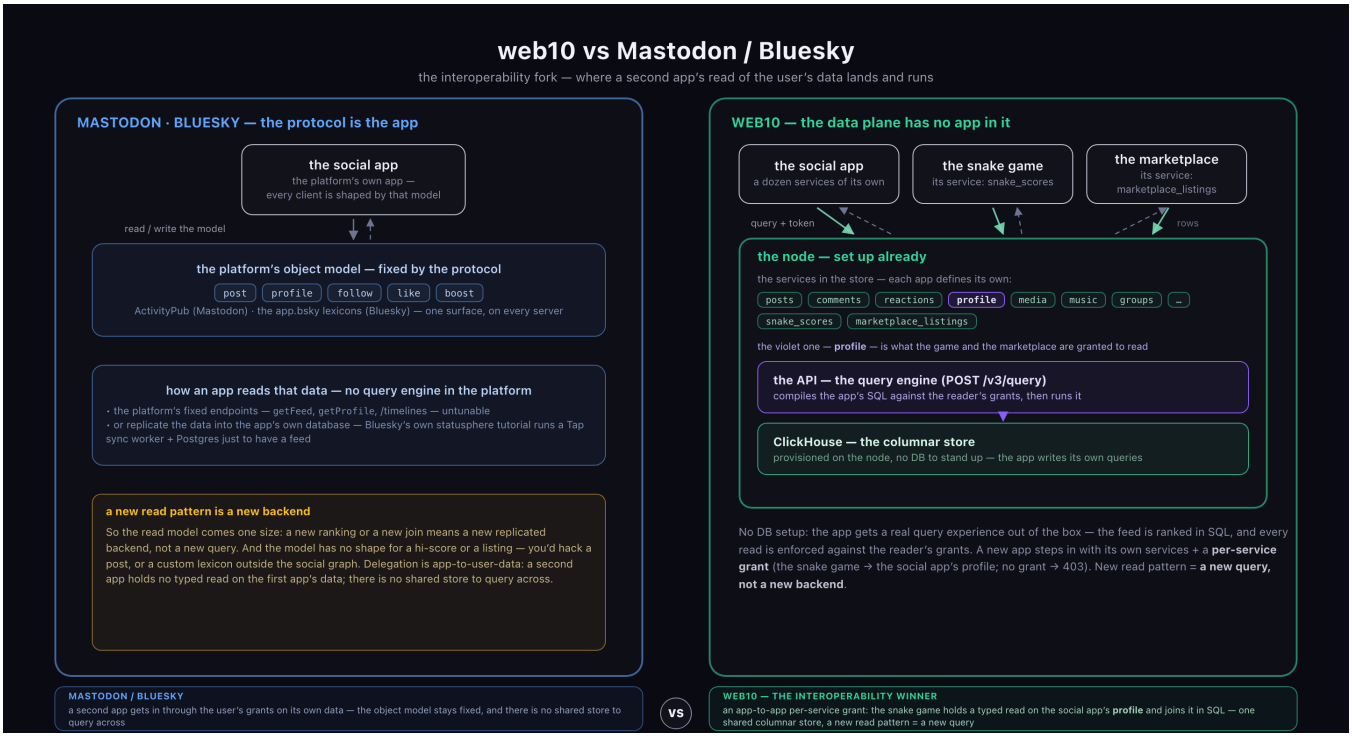
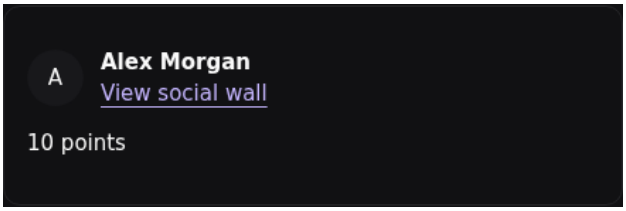


Figure 7: web10 vs Mastodon / Bluesky: the interoperability fork—where a second app's read of the user's data lands and runs. Left: the fixed social object model (post · profile · follow · like · boost); there is no query engine in the platform—the app reads through the platform's fixed endpoints (untunable) or replicates records into its own database (Bluesky's own statusphere tutorial stands up a sync worker and a separate store just to have a feed)—and the model has no record shape for a hi-score or a listing. Right: the web10 node, set up already: the social app's services (`posts` · `comments` · `reactions` · `profile` · `media` · `music` · `groups` · …; `profile` highlighted—the one the interop grant covers) plus the new apps' own (`snake_scores` · `marketplace_listings`); the API in action—the query engine (`POST /v3/query`) compiles the app's query against the reader's grants, then runs it—over the ClickHouse columnar store, provisioned on the node, no database to stand up; a second app steps in through a per-service grant the user gives, and no grant, no read. The strip scores the matchup: web10—the interoperability winner (an app-to-app per-service grant on one shared columnar store; a new read pattern is a new query, not a new backend).

Snake: earned score + social identity



Marketplace: listing + social identity

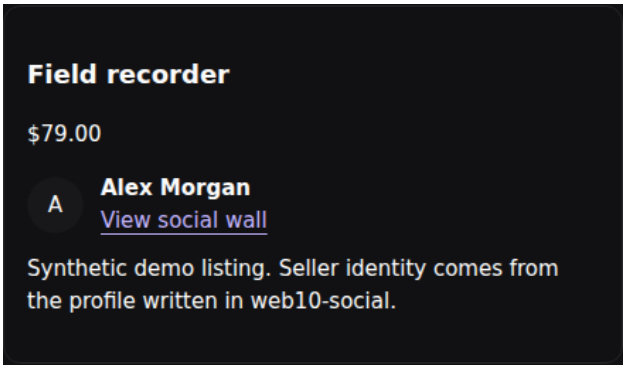


Figure 8: One social-written profile, two application views. Snake joins an earned score to the player's profile; Marketplace joins a listing to the seller's same profile. These are direct browser element captures, with synthetic account and listing data. Full-page, consent, and denial captures are retained in the artifact.

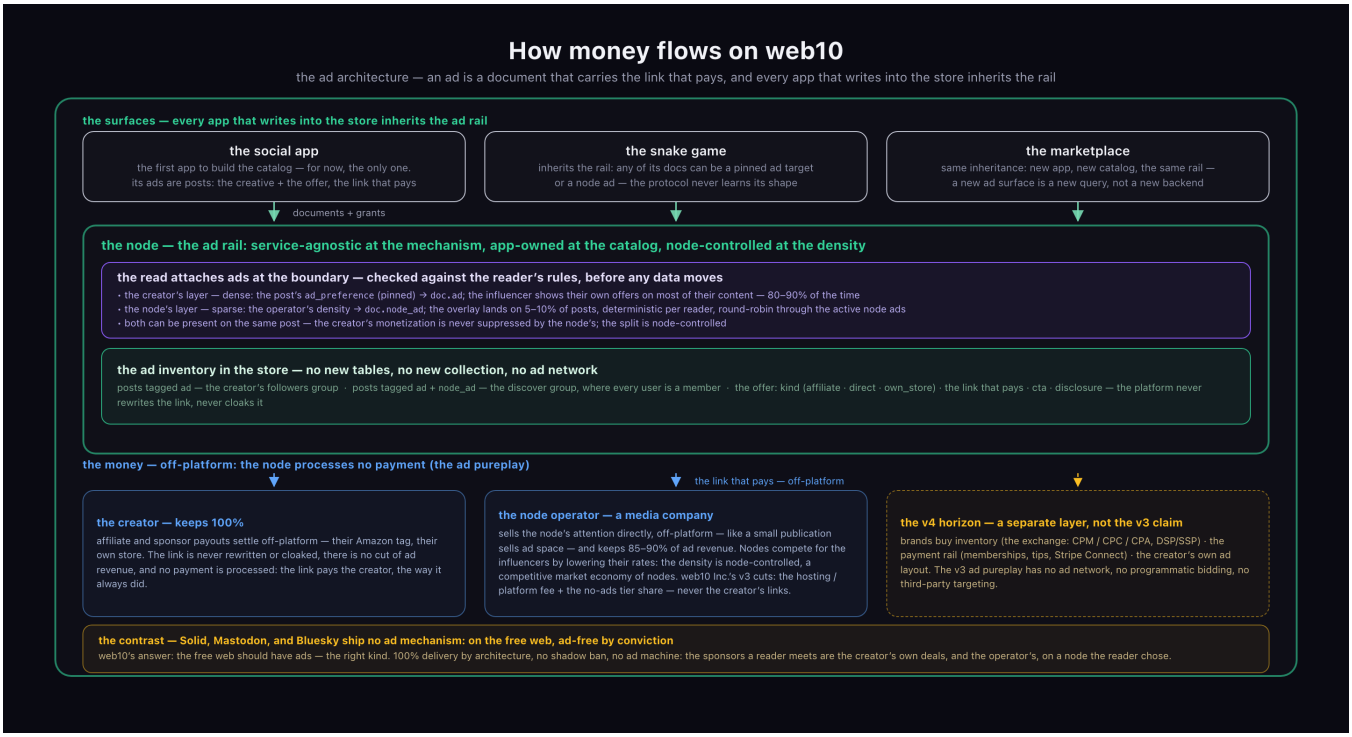


Figure 9: How money flows on web10—the ad rail. Top: every application that writes into the store inherits the rail—web10-social builds the catalog today; the snake game and the marketplace inherit it (a new ad surface is a new query, not a new backend). Middle: the node's rail—the boundary attaches the ads, checked the same way every other read is checked, with the density split explicit: the creator's layer dense (the influencer's own offers on roughly 80–90% of their content) and the node's layer sparse (the operator's overlay on roughly 5–10% of posts, node-controlled, default 10, deterministic per reader); both can be present on the same post, and the node's overlay never steals the creator's monetization. The inventory is tagged documents plus the offer object—kind, the link that pays, call to action, disclosure—never rewritten, never cloaked. Bottom: the money, off-platform—the creator keeps 100% of affiliate and sponsor revenue (no cut of ad revenue, no payment processed); the node operator is a media company who sells the attention directly and keeps roughly 85–90%, and nodes compete for the influencers by lowering their rates. The separate layer names the horizon (the exchange, the payments rail, the creator's own layout)—not this system's claim. The amber strip: Solid / Mastodon / Bluesky ship no ad mechanism—a documented design property; web10's answer: the free web should have ads.

migration and production cross-node integration remain outside the demonstrated scope (§8.6).

7 LESSONS FROM RUNNING IT

The industry track asks what running a system taught you that designing it did not. Four things, in roughly the order they bit.

7.1 The node keeps resisting the killer app

The platform's hardest dependency is its own killer app, and the killer app is always going to ask for bespoke surface. A feed is faster to ship as an endpoint than as a client query; a group's name, description, and banner photo look like exactly the kind of thing a `group_identity` table would hold; a session oracle that knows about the followers group would make the social app's re-authentication path trivial.

We built four such node surfaces, and we removed all four:

- a dedicated feed endpoint, replaced by an application-written query over the services it composes;
- a group-identity table with its own endpoint, replaced by documents in an app-named service read through the normal CRUD and grant path;
- a followers-group check inside the generic session oracle, moved to the one application that knows what a followers group is; and
- an ad path that hard-coded a collection name, replaced with a tag and group membership the applications themselves manage.

The test that caught each one is one sentence: *would a notes app, a music app, or a shop use this endpoint?* The moment the platform API grows a table, function, or field that only makes sense for one application, another application that defines identity differently—or has no followers concept at all—cannot use the platform without the platform knowing its specifics. The cost of holding the line is

real and visible: the client does more work. The feed is a composition the application writes, the followers-group check lives in the social app, and publicness is a grant an application adds rather than a flag a platform flips. The payoff is the result in §6.4 and §8.1: a game and a marketplace, with no relationship to the social application's code, reading its profiles through the same boundary—and no node change when either appeared.

7.2 A self-healing schema raced its own readers

On a fresh stack, the node self-heals its metadata schema with a deferred `ALTER TABLE` at boot, adding columns to `group_contracts` that older databases lack. An early read that lands before the `ALTER` commits fails with an unknown-identifier error, and the failure is easiest to hit exactly where the stack is newest—a clean database, a cold boot, the first request. The incident pins the fix: the schema additions must land synchronously and be guaranteed before the first read of that table, or the columns must exist at table creation, and the regression that guards it boots a fresh database and immediately reads the table. The general lesson is the one that survives the particular bug: a self-heal that may lag is fine; a self-heal that may *race* is not. Test the fresh-boot path, not just the steady state.

7.3 What the shared columnar table costs, day to day

The data-plane choice of §3 trades physical isolation for cross-author composition, and the running platform meets the price of that trade every day: every read pays the version deduplication, because `ReplacingMergeTree` only settles versions when it merges; "deletion" is an insert, so a tombstone is a row that must be ranked, not discarded; merge pressure is background CPU that does not show up in request latency until the host is busy; point queries are paid to an engine built for scans; and every tenant in the node shares the same resources. These are not defects of the design; they are the design's operating envelope, and the reason the query boundary of §5—not a physical partition—is where reader isolation lives.

7.4 The first measurement pointed at the access layer, not the database

The naive assumption is that a columnar engine will make reads cheap, so any latency must live in the engine. The fan-out measurement of §8.4 says the opposite at scale: with a reader following 1,000 creators, the per-group metadata resolver's median readable time was 7.64 seconds while the document scan it fed was already done—the authorization machinery, not the database, was the bottleneck. The batching decision (three metadata queries plus in-memory resolution, §8.4) is the design consequence: the query endpoint should use the batched resolver for large candidate sets. And the honest counterweight is in the same trial: at one candidate group, the per-group path is *faster*, because three fixed round trips dominate when there is one group to check. The measurement that looks like an unqualified win is a tradeoff with a knee; reporting both sides is the whole point of §8.

8 THE EVIDENCE

The evidence is connected, bounded, and named: each part is organized as the question it answers. Does an unrelated application actually reuse another's data through the access boundary (§8.1)? Does a grant reach across the node boundary, and can the owner revoke it locally (§8.2)? Does a public subscription outlive its provider (§8.3)? What does the access layer cost (§8.4)? And is anyone actually using this (§8.5)? The first four use synthetic data on isolated test stacks; the operating point is a read-only observation of the production node. None is a capacity or scale result.

8.1 Connected interoperability

Question: do unrelated applications actually reuse each other's data, through the access boundary, with consent and revocation exercised in a real browser?

Browser reuse. A fresh account logged into web10-social through its authenticator popup and edited its profile name to Alex Morgan. The browser approved Snake's app request, earned and saved ten points, and displayed the joined profile. After app-contract revocation, a new query returned 403 and a leaderboard refresh cleared the result and displayed an access error. After sign-out, Marketplace requested its service permission through the popup, published a listing, and displayed the same profile. Reloading the listing's detail URL preserved that result. API assertions checked exact authors, row counts, profile bodies and listing identity, so the username fallback alone could not satisfy the test.

Independent access controls. A separate HTTP case used three accounts and two social-written profiles. It checked exact two-author score/profile and listing/profile joins. With no app permission for `profile`, both queries were denied; granting it enabled the joins, and revoking the app contract denied them again. A separately labeled private-profile fixture isolated the group gate: membership enabled an exact inner join; removal emptied it while the scores remained readable. The fixture did not replace the social-written profiles in the reuse checks. HTTP contracts were installed through the API; the browser sequence above exercised the actual consent UI.

All three cases—desktop at 1440 pixels, mobile at 375 pixels, and HTTP correctness—passed in the final fresh-stack run on 2 November 2026, with zero retries or skips. Six flow captures and two identity-card captures per viewport document the observed results.

Setup and scope. The run used a four-vCPU, 8-GB DigitalOcean sfo2 host, with ClickHouse 24.8.14.39, Playwright 1.63.0 and Chromium 153. The artifact contains the exact experimental source, configuration, image identities, results and all 16 original captures. The source includes local modifications rather than an unmodified release; the complete snapshot is retained in the interoperability artifact. The API, authenticator and social frontend used development configurations; the marketing frontend used a production build. Schema initialization preceded API startup. Project storage and the droplet were removed after collection.

The hosted apps share one browser origin, although the HTTP case supplies a separate app Origin. They demonstrate reuse across applications, not separate-browser-origin isolation or an

independently recruited developer study. Clients know the profile format; automatic schema discovery is outside this result. Initial-letter avatars are not media-delivery evidence. These controls establish named grant/revocation behavior, not a stale-access or capability-expiry bound.

8.2 Cross-node identity and local revocation

Question: does a grant really reach across the node boundary—and can the owner revoke it locally?

We ran a synthetic correctness scenario across two API processes, separate ClickHouse and MinIO instances, independently generated HS256 signing secrets, and real HTTPS home-issuer certification with an ephemeral test CA. Bob creates a private group and document through B's API. Alice's A-issued JWT initially fails both B's document read and query. This first failure is the one that matters: giving a *distinct B-side Alice* the membership also fails to authorize A's Alice—two Alices, same name, and the node tells them apart only through the qualified principal. Bob then grants the exact foreign principal through the owner API. Both read paths return the exact document; removing that membership denies both, and regrant restores both.

All steps reuse the same JWT, and the certification traffic shows why the model works the way §4 claims it does: A's TLS proxy records exactly one successful certification across this entire lifecycle. B rechecks access on every read while retaining the cached identity answer. A foreign-provider JWT signed with B's key is rejected twice, with one failed issuer call and a negatively cached second rejection. An independent Origin-bearing query gate denies a local namesake's app grant, allows a qualified foreign app-contract fixture, and denies again after its revocation; direct group-authorized reads remain available.

The reported scenario completed without retries on an unmodified source snapshot. It is a correctness experiment, not a deployment or load trial. Docker-internal, globally classified addresses exercise the public-address guard; the test CA and network do not establish public-internet interoperability. Foreign app grant/revocation use labelled service-function fixtures, not browser consent or completed foreign enrollment. Owner grant/removal is not foreign self-join. Exact source, configuration, request outcomes, image identities and cleanup records are retained in the experiment artifact, with earlier attempts documented separately.

8.3 Public domain subscriptions across provider loss

Question: can a public subscription outlive the provider that hosts it?

A separate research client stores only a creator's normalized domain in a private app document at the reader's independent home. It freshly resolves the existing TXT pointer, checks the selected account's verified alias, and reads public posts anonymously. It uses ordinary CRUD/groups and the existing authenticated domain-verification endpoint—not a new node follow API.

On desktop (1440 × 1000) and mobile (375 × 812), the creator logged in, copied/published the TXT, and verified the old account through actual local UDP DNS. The reader clicked Follow once. Then the old API/TLS services were stopped. With the original provider unavailable, the creator retargeted DNS and verified a new account without the old provider's approval. Reload and ordinary reader login displayed new public posts with the identical stored follow ID/body and one total follow creation. No old content was migrated. And the asymmetry that should hold: explicit old private access returned 200 while new-target private access remained 403—the public subscription follows the domain, but private access is not inherited by the new target. Figure 4 draws the move the demonstration exercises; exact panel captures, the authenticated hookup, and the 375px evidence remain in the artifact.

Additional controls delayed real responses at five identity-transition boundaries and verified that logout or account changes prevented stale UI updates. Missing or ambiguous DNS cleared displayed content without deleting the saved subscription, and DNS/alias/public requests carried no credentials. The experiment used an unmodified source snapshot. Exact source and configuration, ten captures, request outcomes and cleanup records are retained in the prototype artifact.

This is a configured two-node client, not deployed social follows or private principal continuity. API app/private grants, TTL-zero local DNS, a loopback HTTP JSON bridge and browser acceptance of test-certificate errors are explicit fixtures. Retained DNS control and available reader/new node/resolver are assumptions; domain sale/theft/expiry and public DNS/TLS/availability are not solved. The result demonstrates bounded public subscription continuity, not absolute unbannability.

8.4 The metadata fan-out cost of the access layer

Question: what does the access layer cost, and where does the cost live?

The access layer resolves the reader's readable groups before the query executes. We measured that cost to understand the design, not to report a latency distribution: the question was whether per-group metadata resolution scales with the number of candidate groups, and what a batched resolution buys. Figure 10 plots the trial; the numbers it carries are stated here, with the scope that bounds them.

Setup. One sequential trial on a DigitalOcean sfo2 `s-4vcpu-8gb` host (four vCPUs, 8 GB nominal RAM, Ubuntu 24.04, Linux 6.8, Docker 29.1.3), co-locating a single Uvicorn process and ClickHouse 24.3.18.7. The synthetic seed (seed 42) created 1,000 creators with one small post each and a reader who joined all their groups; those groups also granted public reads. Personal queries supplied $K = 10$, 100, or 1,000 candidate groups on this same dataset and host; a separate discover query supplied one board group ($K = 1$), containing ten additional posts. Twenty samples per cell, 400 measurements; all 320 B/C HTTP requests returned 200. Queries selected document IDs and bodies, ordered by creation time and ID, with a limit of 20. They did not execute engagement joins, media preparation, or browser page assembly. HTTP requests supplied no Origin header, so this trial does not exercise the browser-origin app-contract gate.

Variants. The per-group resolver performs point metadata queries for each candidate group. The batched resolver fetches memberships, principal-class grants and role definitions across

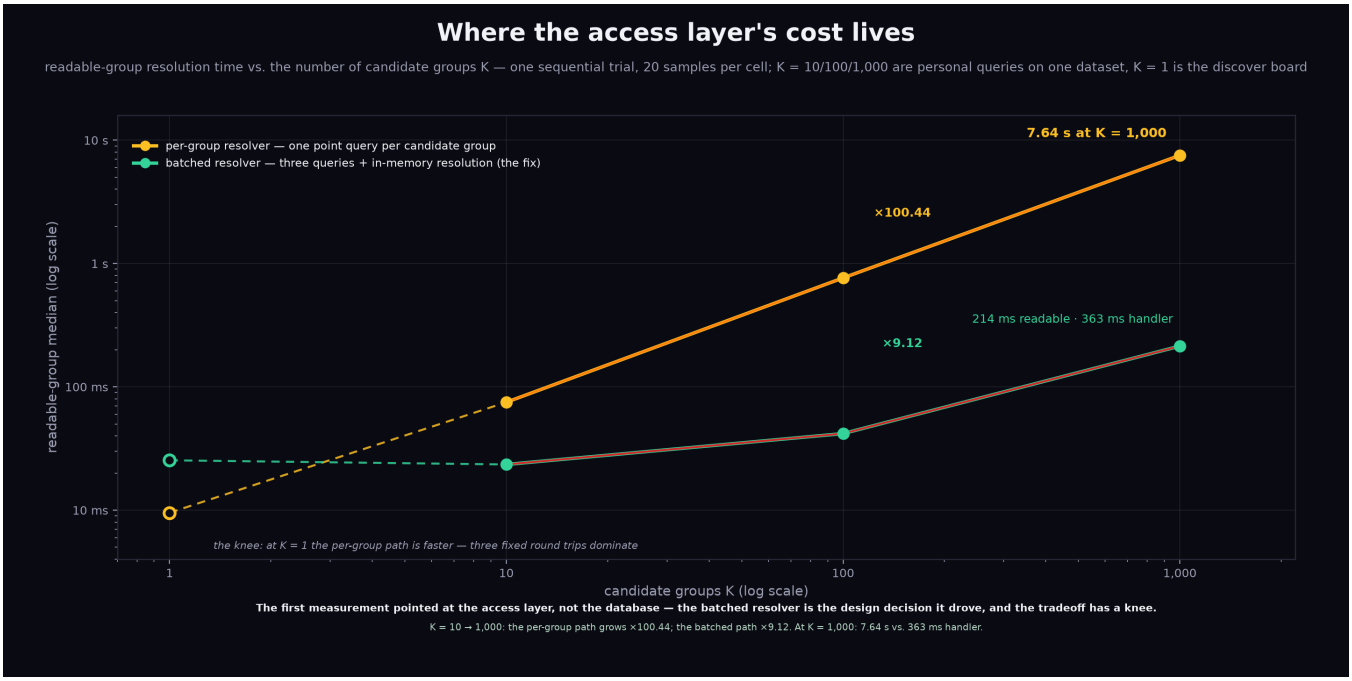


Figure 10: Where the access layer's cost lives: readable-group resolution time versus the number of candidate groups K (log-log), the trial of §8.4. The per-group resolver (amber) grows ×100.44 over $K = 10 \rightarrow 1{,}000$ and reaches 7.64 s at $K = 1{,}000$; the batched resolver (green) grows ×9.12 and sits at 214 ms readable (363 ms handler). $K = 1$ is the discover board—one candidate group, an open marker on the dashed leg—where the per-group path is faster, because three fixed round trips dominate. The knee is the tradeoff; the batched resolver is the design decision the measurement drove (§7).

the candidate set in three database queries, then resolves access in memory. This reduces round trips, not the amount of permission metadata to process. We measured each resolver with real document execution (B) and a fixed 20-row synthetic scan result (C). C retains the policy/compilation path but is not an output-equivalent query execution. A third variant (A) precompiles the known-readable projection and times ClickHouse execution directly, outside the HTTP handler.

Finding. The per-group resolver's cost grows linearly with the number of candidate groups: increasing K from 10 to 1,000 increased median readable time by 100.44× (75.03 ms to 7,535.71 ms; p95 93.30 ms to 8,504.61 ms), against 9.12× for the batched resolver (23.47 ms to 213.98 ms; p95 31.17 ms to 284.03 ms). At $K = 1000$ the batched path is 35.22× faster on readable time and 21.04× faster on handler time (363.31 ms median handler). Stubbing the execution (variant C) keeps the separation: 8,388.39 ms per-group versus 222.57 ms batched at $K = 1{,}000$. The per-group path still completes requests, but its 7.64-second median at $K = 1000$ is a practical bottleneck for a reader who follows many creators; the batched path's 363 ms is not. The direct-scan baseline (A) stayed at 28.6–54.2 ms across $K = 10$–1,000—the document scan was already done while the access layer was still resolving. This is the design decision the measurement drives: the query endpoint should use the batched resolver for large candidate sets.

The tradeoff is honest in both directions. Batching is not uniformly cheaper: at the discover board's $K = 1$, the per-group path is faster (9.53 ms readable versus 25.37 ms batched), because three fixed round trips dominate when there is one group to check. And batching does not make the cost constant: the batched handler median still grew 4.47× over the 100-fold increase in K, as the metadata to process and the compiled query to emit both grow with the candidate set. The result is a characterization of the access layer's cost structure—metadata fan-out is the dominant term at large K, and batching is the mitigation—not a claim of constant latency or a performance benchmark.

The trial is a single sequential run (20 samples per cell, 400 measurements, one host, localhost HTTP, no concurrency, no warm-up exclusion), and the query is a simplified projection without engagement joins, media preparation, or browser page assembly. It measures the access layer's cost structure on a representative workload; it does not establish a latency distribution, a capacity bound, or a causal authorization-overhead estimate. The direct-scan baseline (A) excludes authentication, compilation, and serialization, so $B - A$ is a pipeline-versus-scan comparison, not a matched causal measurement. The trial used seed 42; exact source, configuration and per-sample measurements are retained in the measurement artifact. An earlier 100-creator trial on a different host and source snapshot showed the same qualitative tradeoff and is not pooled with this trial.

8.5 A deployed operating point

Question: is anyone actually using this?

The platform is deployed and serving authenticated users. A read-only observation of the production node on 28 October 2026 recorded 597 account rows, 593 live documents across 30 distinct authors and 10 services, 49 group contracts, and 1,298 membership rows, on a host running the social, authenticator, and API frontends. Nine authenticated browser navigations (following feed, music, groups) returned HTTP 200 with a retained session and zero uncaught page errors; the following feed rendered 20 posts in a median 5,396 ms of full-page readiness. This is an operating point—evidence the system runs and serves real users—not a capacity or scale result: the host also runs a development stack, the observation is a snapshot without an activity window, and the readiness figures include frontend startup and network work, not server-query duration. The 5.4-second figure is the kind of number the industry track exists to report: it is real, it is bad enough to matter, and the full-feed measurement named in §8.6 is the dissection it asks for.

8.6 Scope and open work

The evidence above is bounded by construction, and the open work is specific.

Federation beyond the correctness slice. The isolated experiment in §8.2 exercises home-issuer certification and owner-managed foreign grant/removal, not a complete federated product. The domain prototype adds public subscription continuity, not production social integration or private-principal portability. Independent production signing-key provisioning, complete local-principal migration, foreign self-join and app enrollment, multiworker/restart behavior, public deployment and client-side feed merging remain open. Cached identity answers intentionally last until JWT expiry; this does not establish immediate issuer-side revocation or revoke already-issued media capabilities. JWKS would be an optional verification optimization, not a prerequisite for the live-call model.

Workload and baseline. The metadata fan-out measurement is a single sequential trial on a simplified query. Full-feed latency (including engagement joins, media preparation, and browser page assembly), concurrent capacity, mixed allowed/denied access, and a matched shared-table policy baseline (PostgreSQL with row-level security, same data and access semantics) remain to be measured. A causal authorization-overhead estimate requires an isolated ablation with equivalent preauthorized result sets, not the pipeline-versus-scan comparison reported here.

Correctness. Security evidence consists of regression tests and the named connected controls. Whole-system conformance, revocation-latency bounds (racing grant changes with reads through metadata visibility, in-flight requests, and issued capabilities), and the media capability boundary remain open. Direct read-URL refresh enforces the verified caller's exact owner namespace; that check alone does not establish stored-reference provenance or revoke previously issued capabilities. Parser/engine semantic differences and result-projection provenance require targeted testing.

9 RELATED WORK

The contribution is an integration of established ideas—user-addressed storage, user-managed grants, and application-defined queries over authorized relations. The two comparison figures of §3 and §6.4 organize the matchup visually; what follows is positioning, not a survey. We discuss design differences from the cited systems rather than claiming that no prior implementation combines these capabilities.

Decentralized social data. ActivityPub [11] (Mastodon) defines client/server and federated server/server activity delivery; its authorization is platform-defined—follow/block and collection-level visibility—not per-principal grants over application-defined services. ATProto [10] (Bluesky) separates repositories, relays, and AppViews, supporting user agency and provider switching; its visibility model is likewise collection-level, with AppViews providing application-defined *reads* but not user-managed *grants*. These protocols organize access around their social application models. web10 instead exposes service-scoped roles to users and accepts application-defined queries over the resulting authorized relations—the seam that §6.4 exercises with two applications that share nothing but the document convention. Figure 7 draws the seam.

Solid: the closest prior work. Solid [15] separates applications from personal data stores (Pods), with social application work [14] and a maintained Community Solid Server [7]. WAC supports resource permissions and group agents through `acl:agentGroup` [5], and ACP adds agent, client, and issuer matching with allow/deny mode sets [3]. web10 does not claim Solid lacks group or client authorization; it has both. The distinction is in the *shape* of the model. Solid's access control is *resource-centric*—ACLs attached to linked-data resources and containers, evaluated per resource. web10's is *service-centric*—a uniform role/grant vocabulary (principal class × service × operation) in which a role is a per-service permission map, the same shape across the app-trust and person-trust layers, and the grant is a row naming a globally addressable principal. Concretely: Solid answers "who may read or write this *resource*?"; web10 answers "what *operations* may this *principal class* perform on this *service*, in this *group*, under this *app contract*?" The latter is the IAM policy shape (principal, service, operation, grant) exposed to the end user, not an infrastructure operator. A second distinction is the query surface: Solid's cross-Pod query cost depends on deployment, caching, and linked-data traversal strategy; web10 composes application-defined SQL over a shared columnar store with the access decision made on the input (§5). The fork is drawn in Figure 2. A performance comparison would require a matched workload and deployment; we do not claim one here.

Cloud IAM and relationship-based authorization. AWS IAM (documentation) [1] and the RBAC/ABAC literature [6, 8, 16] provide the policy concepts web10 adapts at the user boundary: principals, roles, service-scoped operations, revocable grants. The move is directional—a familiar policy vocabulary exposed to *users* about their own content and groups, orchestrated by the API and query layer, rather than to infrastructure operators about cloud resources. At industrial scale, Zanzibar [12] is the closest prior art in *relationship-based* authorization: a uniform ACL data model

and configuration language across hundreds of services, with consistency as a first-class problem (causal ordering of ACL updates, snapshot reads). Zanzibar's model is *infrastructure-side*—the operator defines the relationship schema and the service enforces it, while applications can expose sharing controls to their users. web10 explicitly exposes service-scoped grants as user-managed data, names globally addressable principals, and enforces access at the query layer over a shared columnar store. Zanzibar's consistency guarantees motivate the explicit revocation-visibility evaluation we leave as forward scope (§8.6); we do not claim equivalent consistency here. TAO [4] provides the deployed precedent for *read-time* privacy checking at social-graph scale (a billion reads/sec, privacy check per item, pull not push); web10's query boundary is the same idea—every row access-checked on its way out—applied to service-namespaced documents in a columnar store, with the check expressed as a reader-filtered CTE rather than a per-item predicate.

Multi-tenant isolation and row-level security. The database literature on tenant isolation—shared tables with an organization identifier (Force.com [18]), schema-mapping techniques [2], and the shared-versus-separate spectrum [9]—is the closest systems ancestor. web10's key-space per user takes that spectrum to its limit: not one tenant per organization, but one logical tenant per end user, in a shared columnar store (§3). Authorship uses `author_key`; shared reads use authorized group relations rather than a physical database boundary. PostgreSQL row-security policies [13] are the alternative enforcement location: per-row policy evaluation inside the database engine. web10 places enforcement above the engine—the reader-filtered CTE is constructed in the application layer before the query reaches ClickHouse [17]. Both placements can combine authorized rows with application-defined joins and aggregation; a matched policy and workload comparison (§8.4) can distinguish the costs of these placements. We report the design difference here and leave the cost comparison as forward scope.

Monetization, and the market the data plane makes. Solid, Mastodon, and Bluesky ship no ad mechanism. That is a documented design property of each—the free-web camp—not an oversight, and the gap stays open until the primary sources are pinned for this section. On those platforms the creator's revenue path is the platform's cut, and the rent is measurable: reach throttling, demonetization, and deplatforming—one million subscribers, three hundred thousand delivered. web10's answer is the ad rail of §6.5 (Figure 9): the creator keeps 100% of affiliate and sponsor revenue, the node operator is a media company that sells the node's own attention, and the platform takes no cut and processes no payment. Figure 11 draws the consequence: because the follow is the fan's own data, bound to the domain (§4), an influencer changing nodes is a rematch, not a loss—and because the overlay density is node-controlled, the market between influencers and nodes is a matching market on *terms* (price, bans, overlay, moderation style, the self-hosted floor as the outside option that keeps the prices honest). The competition is on terms, not on leverage.

10 CONCLUSION

Email decentralized where your data goes; fifty years of practice show that the model works at global scale, for one format, with the access layer left behind. web10 is the generalization: a user-addressed network in which applications define their own document services over a shared columnar store in each sovereign node, in which a uniform, user-managed access model lets any principal on any node hold a role in a user's groups, and in which the query layer supplies the reader's service relations before joins and aggregation execute. The cross-node boundary is the HTTP API and a verified token—remote data is content, never control—and the client, not a server-to-server relay, mediates the reads.

The evidence is connected, bounded, and named. A game and a marketplace reuse the social application's profiles with no node change and no API agreement, with browser consent and cross-author app/group controls exercised end to end; a two-node experiment shows a foreign principal granted, reading, revoked, and restored under a single cached certification; a browser prototype keeps a public subscription alive after its provider goes dark—while the private grants correctly stay behind. And the metadata fan-out measurement identifies where the access layer's cost actually lives at scale: at 1,000 candidate groups, batching reduced median handler time from 7.64 s to 363 ms, while costing more at one group. This is a design decision, not a scalability claim—and it is the kind of concrete, measured tradeoff an operating system earns.

The labor is the discipline: the generic core was earned, not designed from the top, and the four built-and-deleted node surfaces of §7 are the visible part of the 568,594 deleted lines. And the market the data plane makes is why the reach is worth it: the creator keeps their own monetization, and the nodes compete for the creators on terms, not on leverage.

The forward scope is specific: federation beyond the isolated correctness slice (§8.2), full-feed and concurrent workload measurements, a matched shared-table policy baseline, and revocation-latency bounds. The architectural contribution is the integration itself: the email model, extended so that a user's permissions— not just their data—leave the building with them.

REFERENCES

[1] Amazon Web Services. 2025. AWS Identity and Access Management (IAM) User Guide. Amazon Web Services documentation. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

[2] Stefan Aulbach, Torsten Grust, Dean Jacobs, Alfons Kemper, and Jan Rittinger. 2008. Multi-Tenant Databases for Software as a Service: Schema-Mapping Techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. ACM, 1195–1206.

[3] Matthieu Bosquet. 2022. Access Control Policy (ACP). W3C Solid Community Group, Editor's Draft. https://solid.github.io/authorization-panel/acp-specification/

[4] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC '13)*. USENIX Association, 49–60.

[5] Sarven Capadisli. 2022. Web Access Control. W3C Solid Community Group, Version 1.0.0-cr-1. https://solidproject.org/TR/wac

[6] David F. Ferraiolo and David R. Kuhn. 1992. Role-Based Access Controls. In *Proceedings of the 15th National Computer Security Conference*. 554–563.

[7] Joachim Van Herwegen and Ruben Verborgh. 2024. The Community Solid Server: Supporting Research and Development in an Evolving Ecosystem. *Semantic Web* 15, 6 (2024), 2597–2611. https://doi.org/10.3233/SW-243726

[8] Vincent C. Hu, David F. Ferraiolo, David R. Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technical Report SP 800-162. National Institute of Standards and Technology. Includes updates as of August

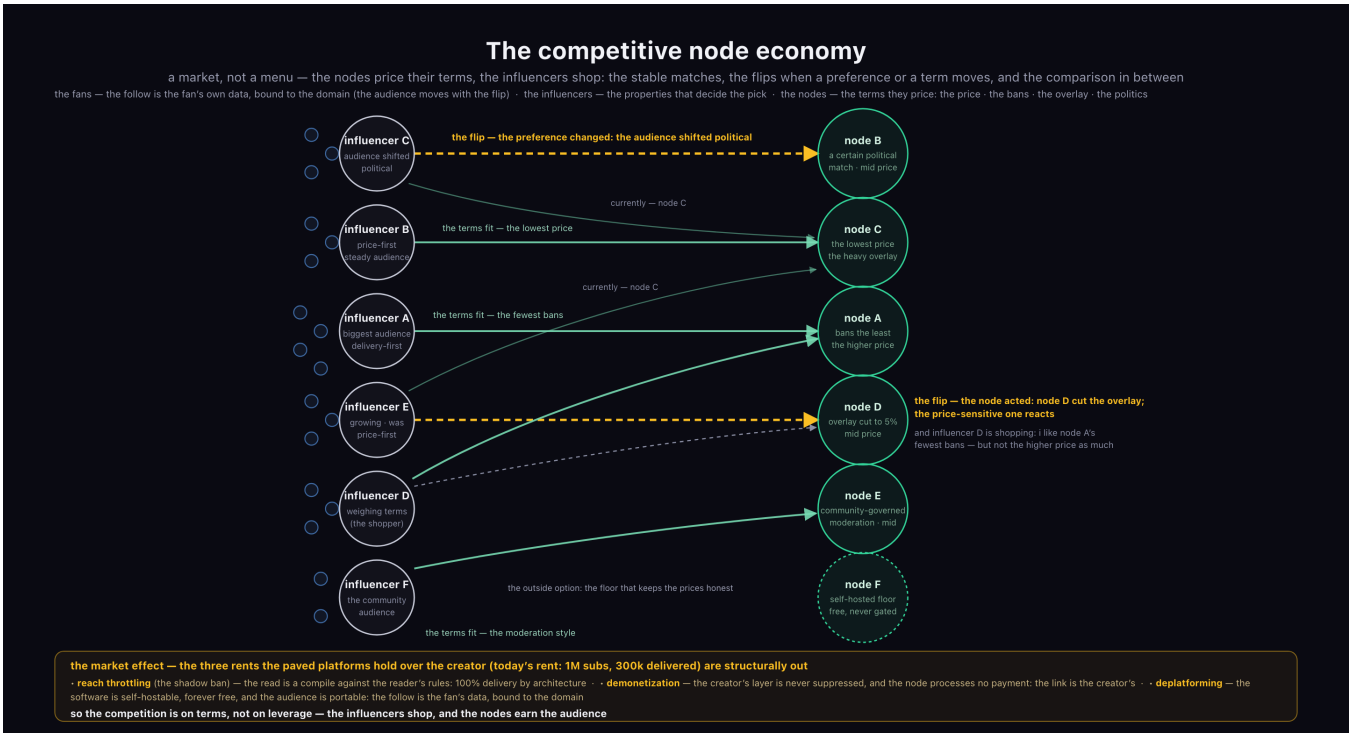


Figure 11: The competitive node economy: a matching graph, 6 × 6. Six influencers (left, neutral) matched against six nodes (right, green), each vertex carrying the market properties that decide the pick. The nodes price their own terms: A bans the least, the higher price · B a certain political match, mid price · C the lowest price, the heavy overlay · D the overlay cut to 5%, mid price · E community-governed moderation, mid price · F the self-hosted floor— free, never gated, the isolated outside option that keeps the market's prices honest. The influencers carry their own properties (biggest audience / delivery-first · price-first · audience shifted political · the shopper · growing, was price-first · the community audience) with the fan dots by each: the follow is the fan's own data, bound to the domain—a flip is a rematch, not a loss. The edges carry the dynamics: the stable matches; two flips—one because the preference changed, one because the node acted; the shopping leg; and node C losing two of its three, because price alone does not hold the audience. The amber strip: the three rents the paved platforms hold over the creator—reach throttling, demonetization, deplatforming—structurally out; so the competition is on terms, not on leverage (§6.5).

2019.

[9] Dean Jacobs and Stefan Aulbach. 2007. Ruminations on Multi-Tenant Databases. In *Proceedings of BTW 2007*. 514–521.

[10] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24)*. ACM, 9. https://doi.org/10.1145/3694809.3700740 Extended version: arXiv:2402.03239.

[11] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/ Editors: Christine Lemmer-Webber and Jessica Tallon.

[12] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association. https://www.usenix.org/system/files/atc19-pang.pdf

[13] PostgreSQL Global Development Group. n.d. PostgreSQL Documentation: Row Security Policies. PostgreSQL documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html Accessed October 2026.

[14] Andrei Sambra, Amy Guy, Sarven Capadisli, and Nicola Greco. 2016. Building Decentralized Applications for the Social Web. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW '16 Companion)*. ACM, 1033–1034.

[15] Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. *Solid: A Platform for Decentralized Social Applications Based on Linked Data*. Technical Report. MIT CSAIL and Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

[16] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *IEEE Computer* 29, 2 (1996), 38–47.

[17] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744. https://doi.org/10.14778/3685800.3685802

[18] Craig D. Weissman and Steve Bobrowski. 2009. The Design of the Force.com Multitenant Internet Application Development Platform. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. ACM, 889–896. https://doi.org/10.1145/1559845.1559942